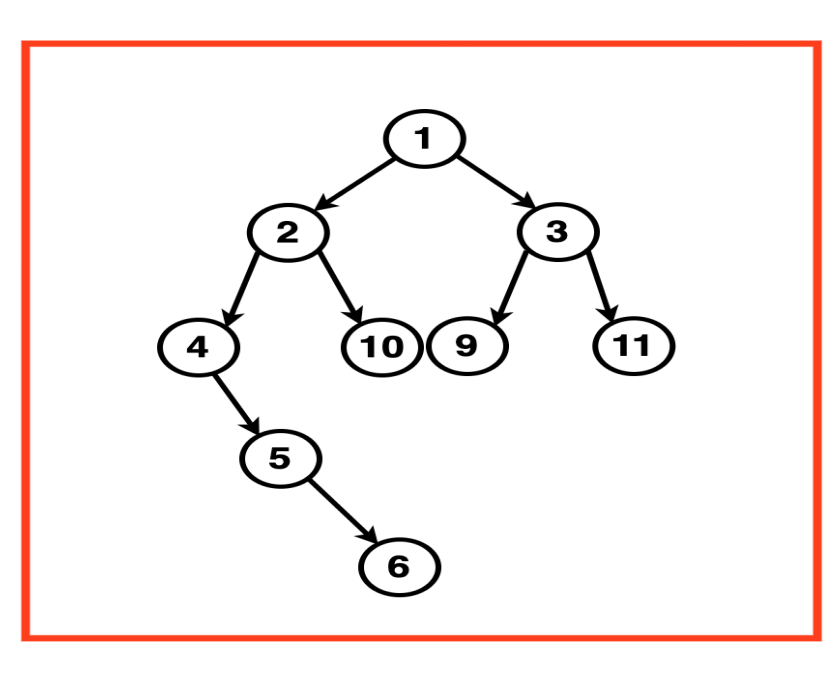
Lec-26

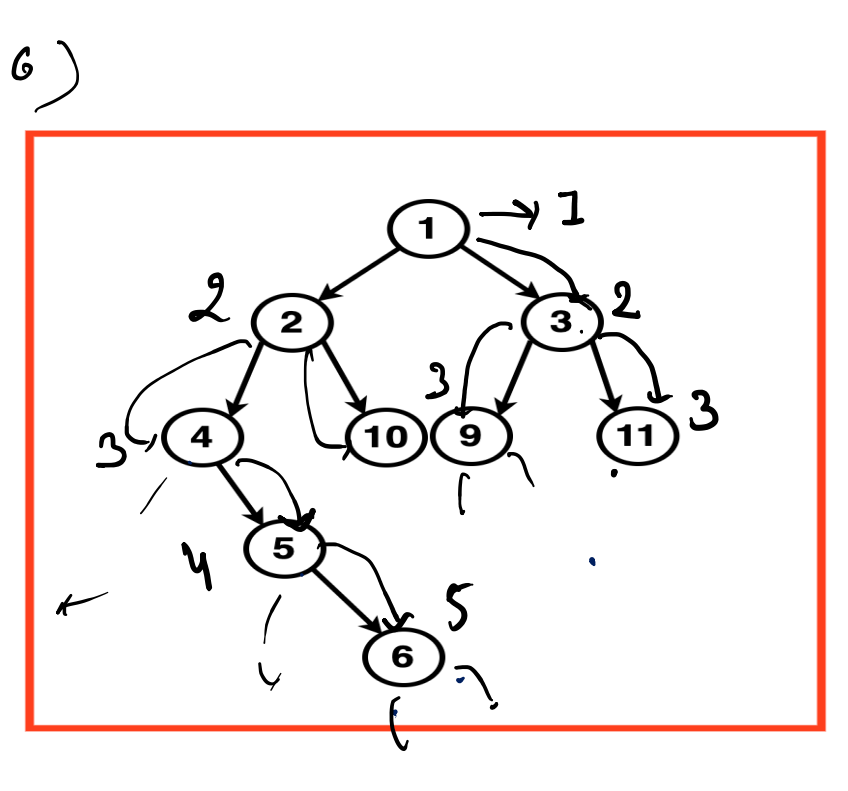
24/08/25 9:07 AM


1
2
3
4
10
9
11
5
6

Left → ✓
Right → ✓
Top
bottom → Map

2 | 3
4 10 9 | 11
5
6

[1, 3, 11, 5, 6]


1
2
3
4
5
10
9
11
6

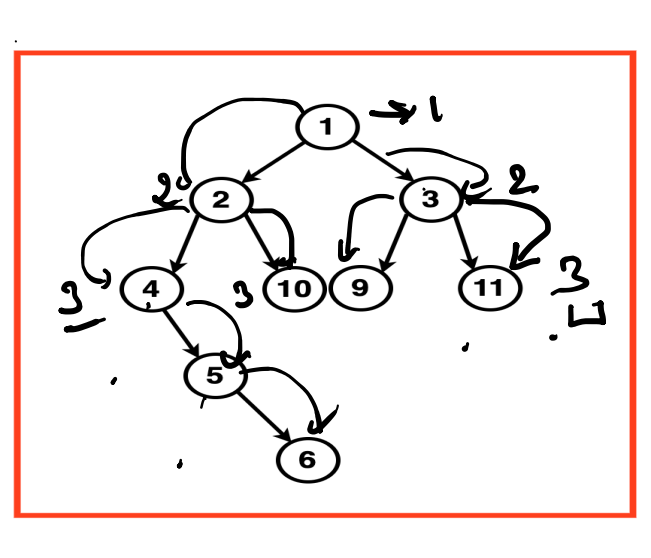
maxdepth = ~~0~~ ~~1~~ ~~2~~ 3 4 5

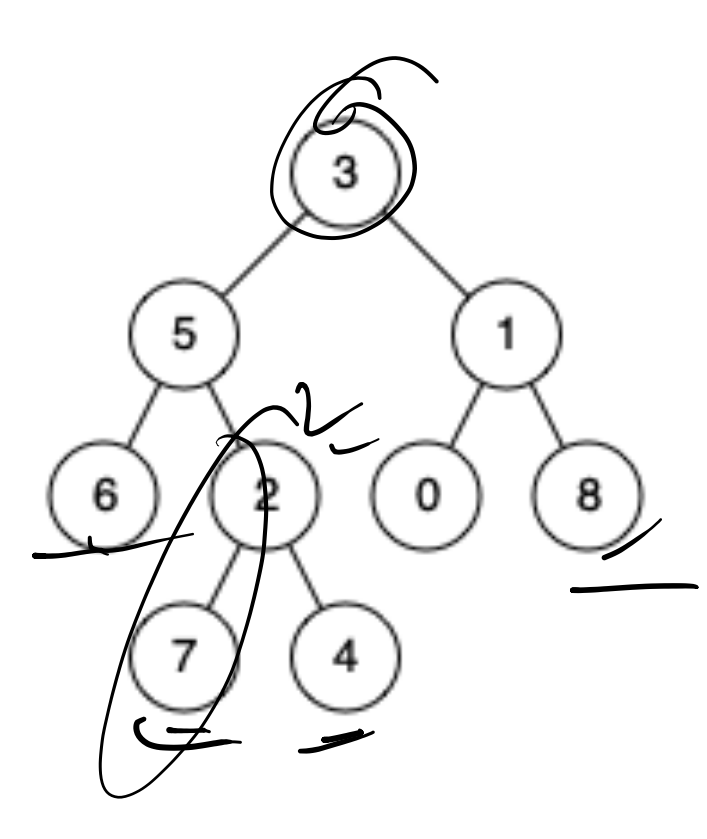
if(maxdepth < currlevel){
maxdepth = currlevel
}

```
class Solution {
    int max_depth = 0;

    public List<Integer> rightSideView(TreeNode root) {
        List<Integer> ll = new ArrayList<>();
        rightView(root, ll, 1);
        return ll;
    }

    public void rightView(TreeNode root, List<Integer> ll, int curr) {
        if (root == null) {
            return;
        }
        if (max_depth < curr) {
            ll.add(root.val);
            max_depth = curr;
        }
        rightView(root.right, ll, curr + 1);
        rightView(root.left, ll, curr + 1);
    }
}
```

max = ~~0~~ 1 2 3 4 5

[1, 3, 11, 5, 6]

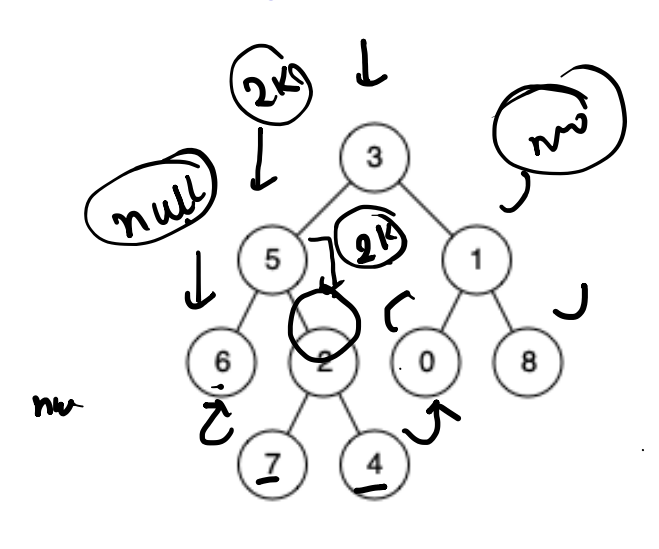

1
2
3
4
10
9
11
5
6



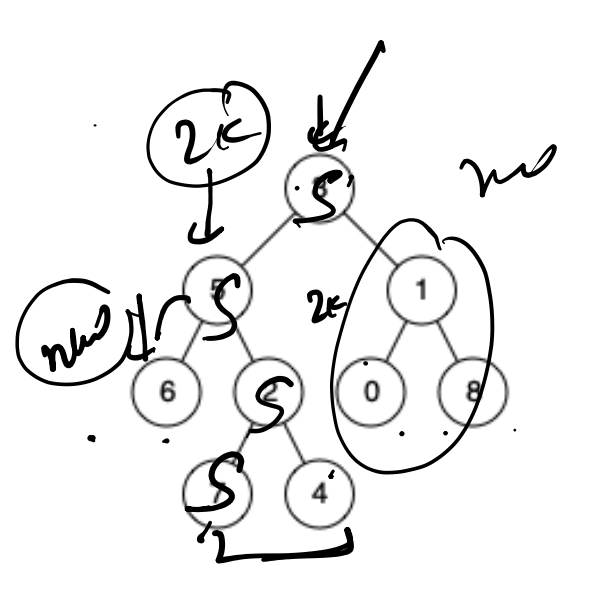
3 5 2 7
3 5 2 4

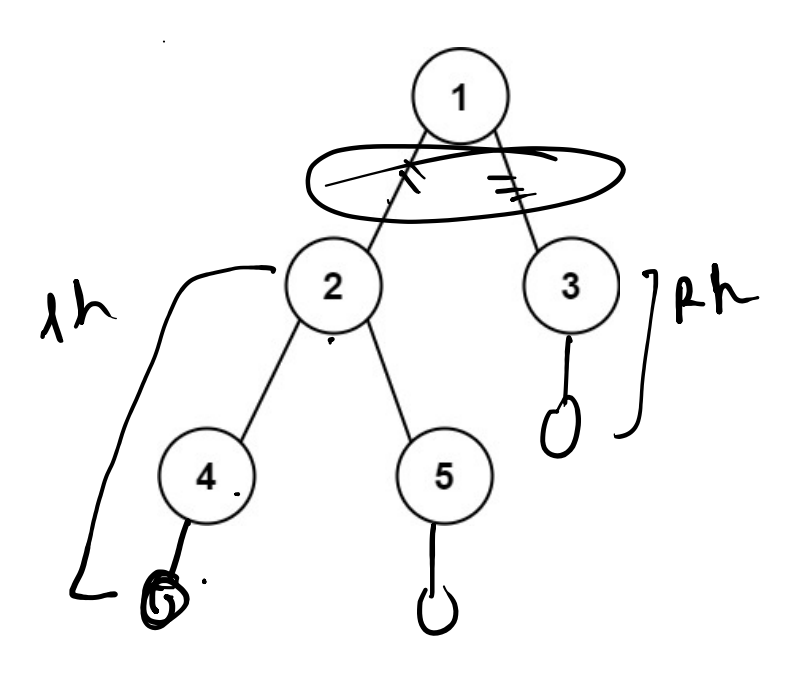
3 5 2
3 5 2 7

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null) {
            return null;
        }
        if (root == p || root == q) {
            return root;
        }

        TreeNode left = lowestCommonAncestor(root.left, p, q);
        TreeNode right = lowestCommonAncestor(root.right, p, q);

    }
```


2k
null
3
5
2k
1
6
2
0
8
7
4

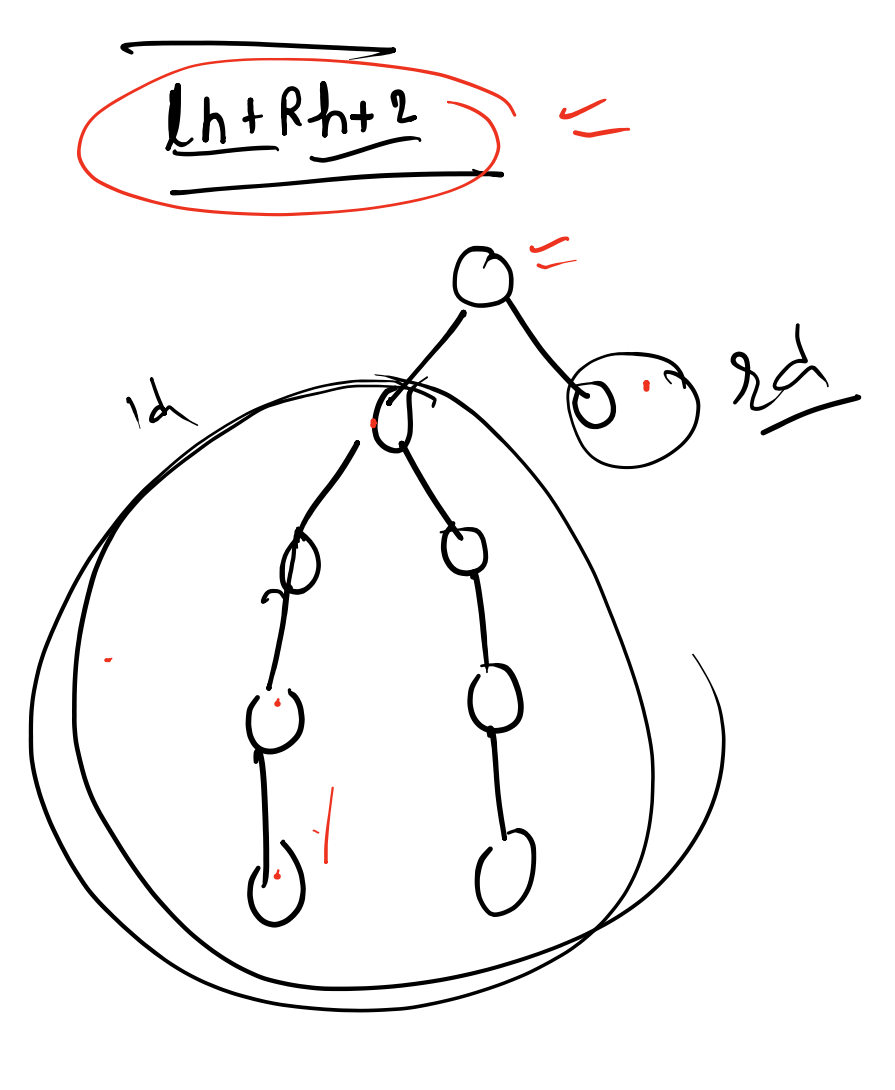
```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null) {
            return null;
        }
        if (root == p || root == q) {
            return root;
        }

        TreeNode left = lowestCommonAncestor(root.left, p, q);
        TreeNode right = lowestCommonAncestor(root.right, p, q);
        if(left!=null && right!=null) {
            return root;
        }
        else if(left==null) {
            return right;
        }
        else {
            return left;
        }
    }
```


2k
5
null
6
5
1
0
8
5
4



2+1+2= 5

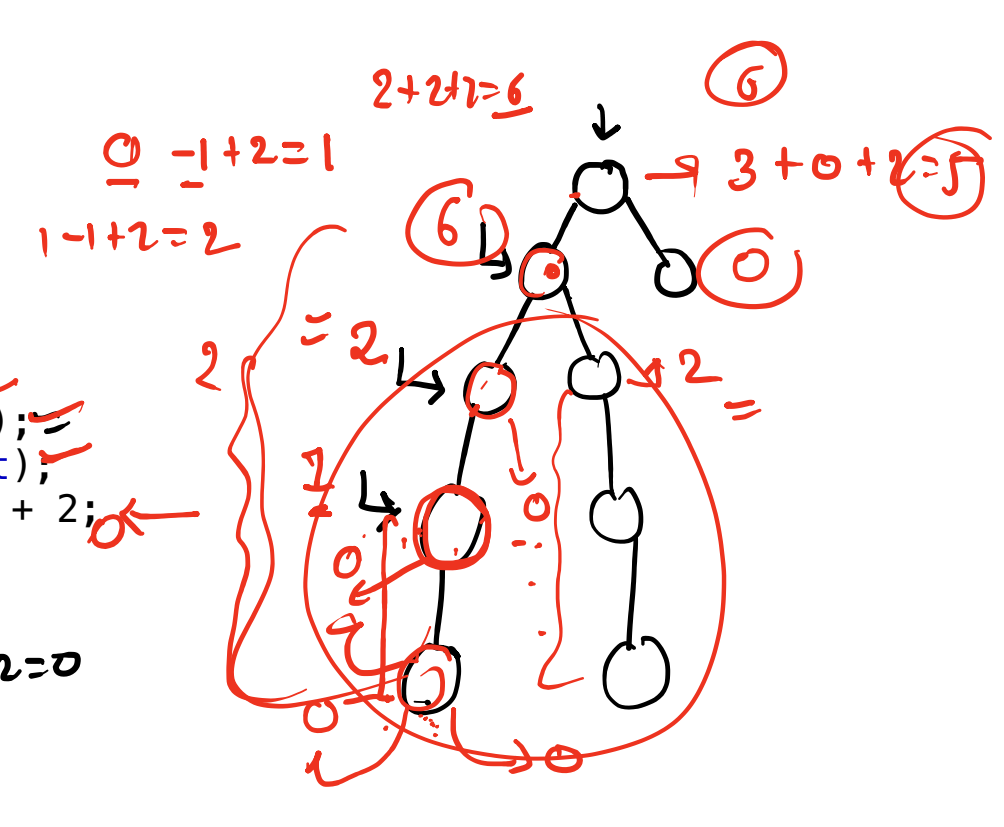
lh+Rh+2

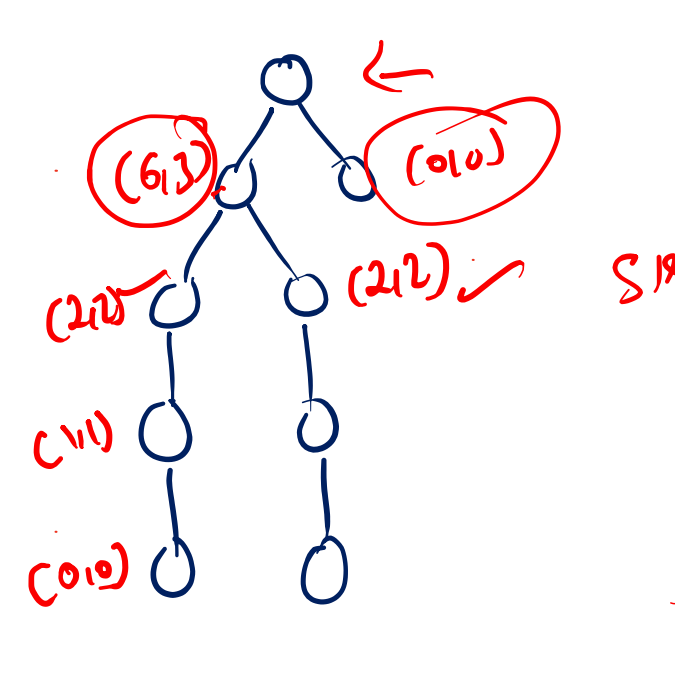


3+0+2=5

```
public int diameterOfBinaryTree(TreeNode root) {
        if (root == null) {
            return 0;
        }

        int ld = diameterOfBinaryTree(root.left);
        int rd = diameterOfBinaryTree(root.right);
        int sd = ht(root.left) + ht(root.right) + 2;
        return Math.max(sd, Math.max(ld, rd));

    }
    private int ht(TreeNode node) {
        if (node == null) {
            return -1;
        }

        int lh = ht(node.left);
        int rh = ht(node.right);
        return Math.max(lh, rh) + 1;

    }
```

-1-1+2=0


2+2+2=6
0 -1+2=1
1-1+2=2
6
6
3+0+2=5
0
2
2
2
1
0
0


(6,3)
(0,0)
(2,2)
(1,1)
(0,0)

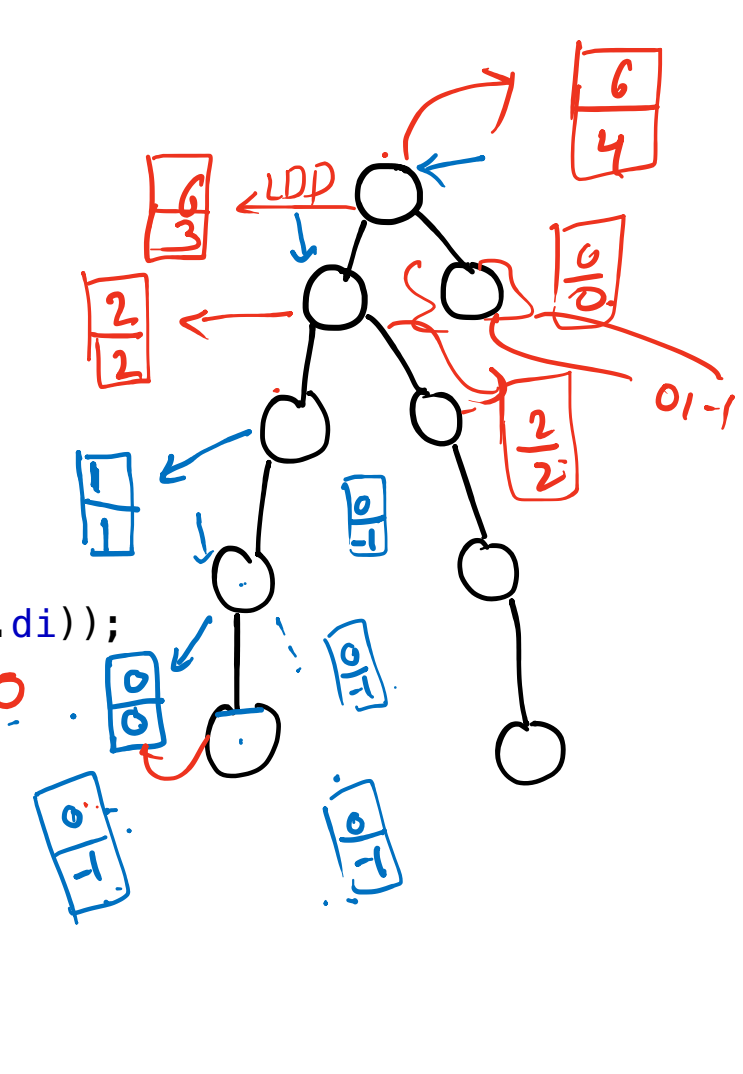
class Student{
String name
int rn

}
Student s = new Student()
rn (8)

```
public DiaPair diameter(TreeNode root) {
        if (root == null) {
            return new DiaPair();
        }
        DiaPair ldp = diameter(root.left);
        DiaPair rdp = diameter(root.right);
        int sd = ldp.ht + rdp.ht + 2;
        DiaPair sdp = new DiaPair();
        sdp.ht = Math.max(ldp.ht, rdp.ht) + 1;
        sdp.di = Math.max(sd, Math.max(ldp.di, rdp.di));
        return sdp;
    }
```

3+0+2=5

5 6


6
4
6
3
LDP
2
2
0
0
0,-1
0,-1
1
1
2
2
0
0
0
-1
0
-1
0
-1

Binary Search Tree

{Binary Tree
Binary Search Tree}

B.T → B.S.T

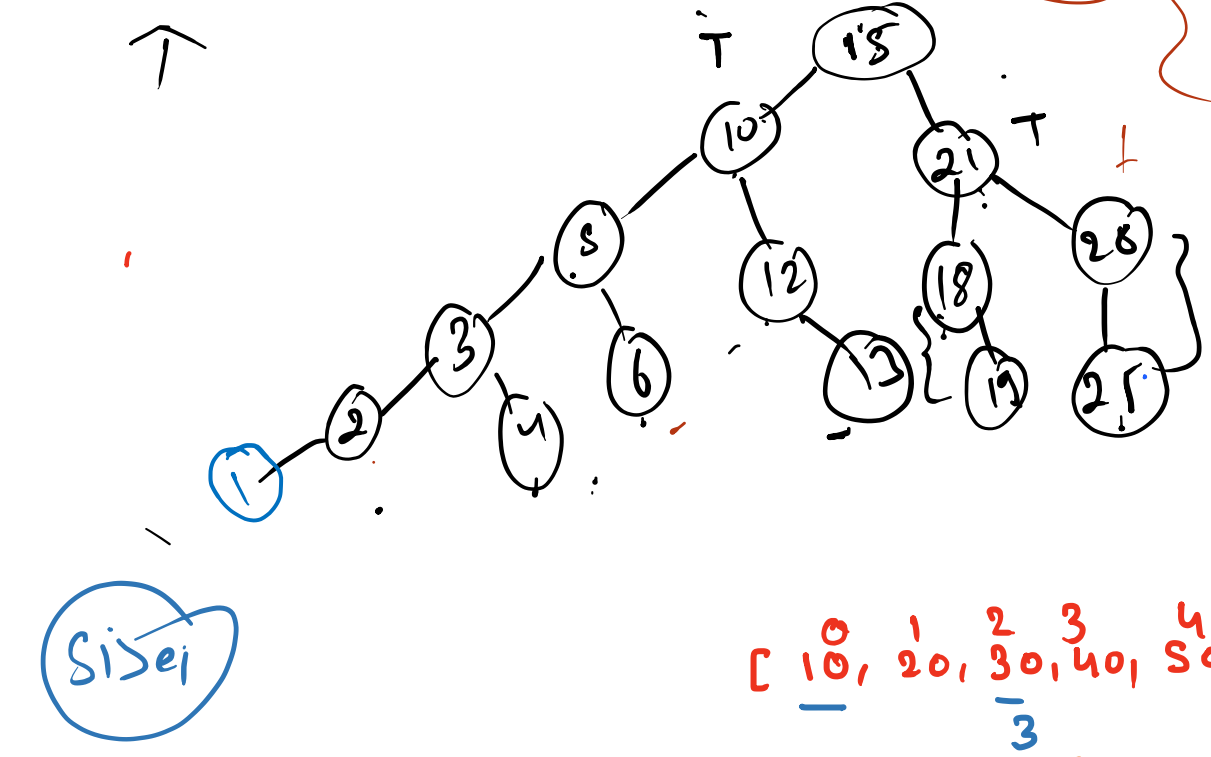
Left max < root
root < right min

Node

In-order Left Root Right

B.S.T

{1 2 3 4 5 6 10 12 13
15 18 19 21 25 28}


T
15
10
21
T
5
12
18
28
3
6
13
19
25
2
4
1
SiSei

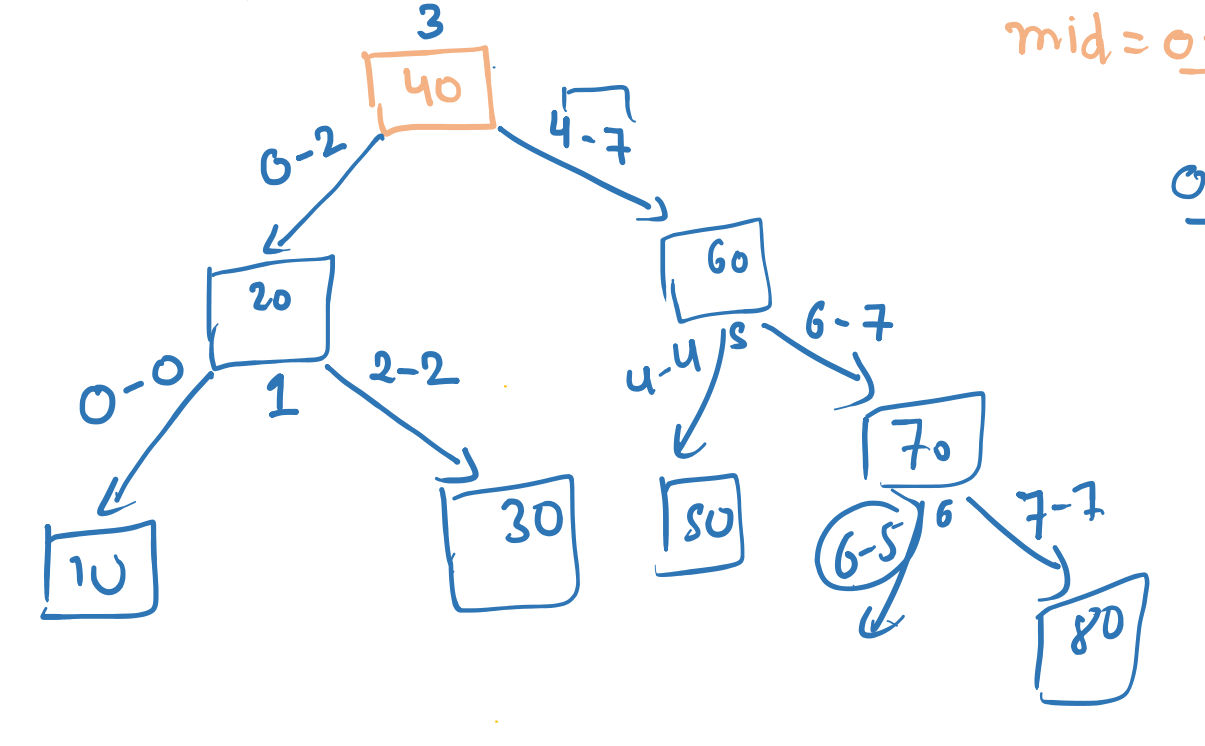
[10, 20, 30, 40, 50, 60, 70, 80]
0 1 2 3 4 5 6 7

4+7/2 = 5

mid = 0+7/2 = 3

0+2/2 = 1

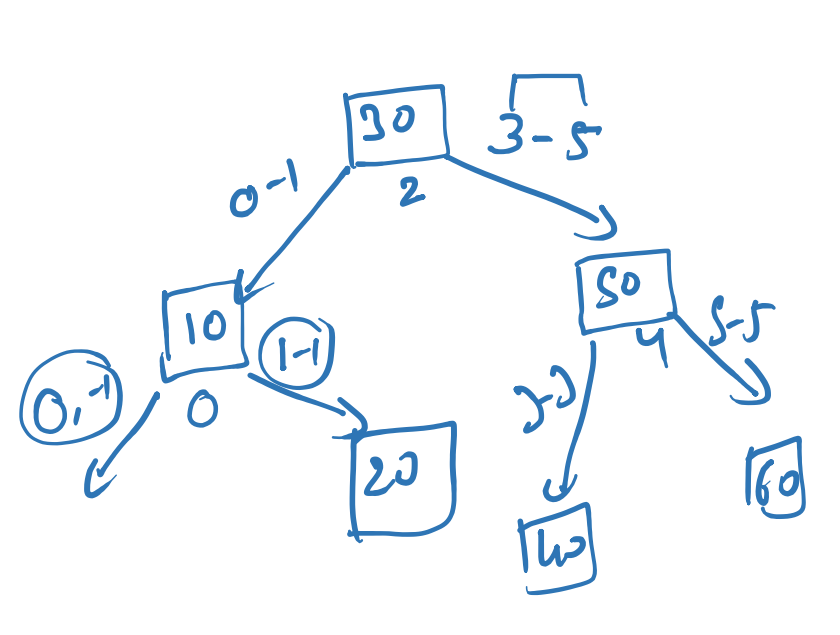
6+7/2 = 6


3
40
0-2
4-7
20
60
0-0
1
2-2
4-4
5
6-7
70
10
30
50
6-5
6
7-7
80

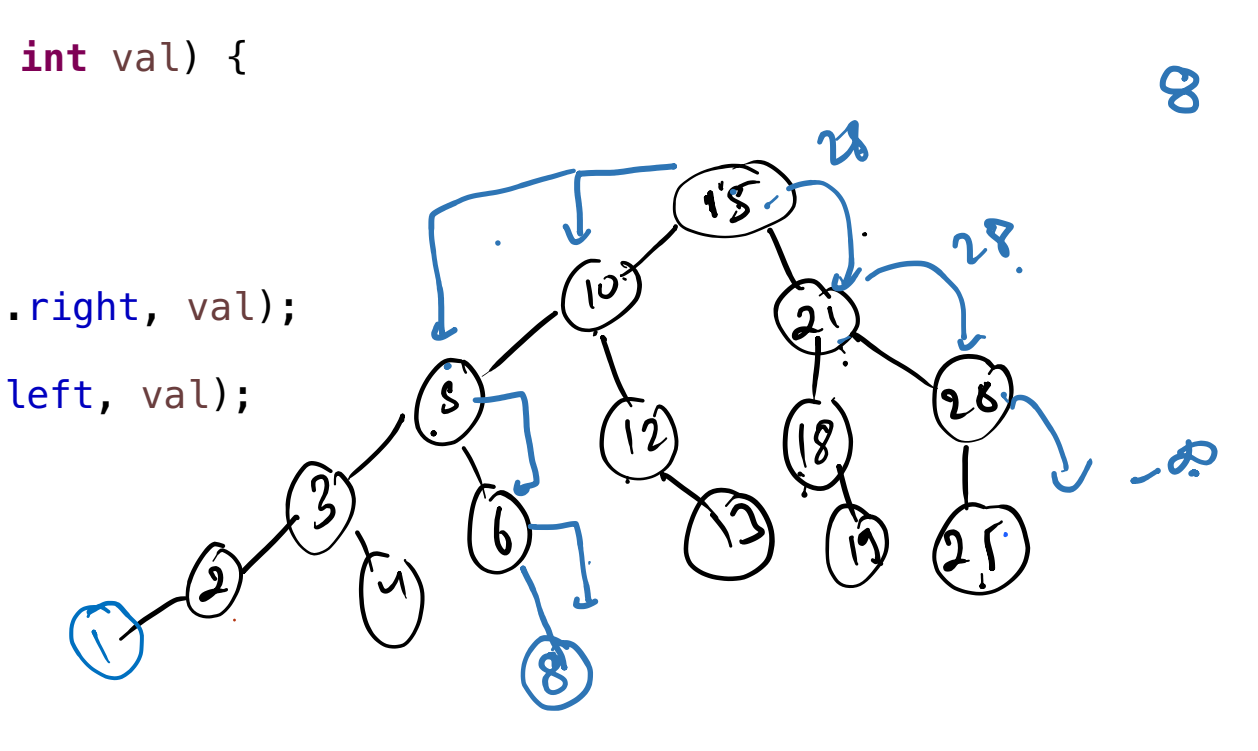
```
public BinarySearchTree(int[] in) {
        // TODO Auto-generated constructor stub
        root = CreateTree(in, 0, in.length - 1);
    }
    private Node CreateTree(int[] in, int i, int j) {
        // TODO Auto-generated method stub
        if (i > j) {
            return null;
        }
        int mid = (i + j) / 2;
        Node nn = new Node(in[mid]);
        nn.left = CreateTree(in, i, mid - 1);
        nn.right = CreateTree(in, mid + 1, j);
        return nn;
    }
```

0+1/2 = 0-1 = -1

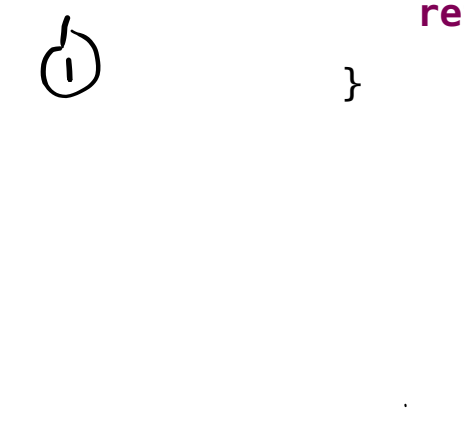
0 1 2 3 4 5
[10, 20, 30, 40, 50, 60]

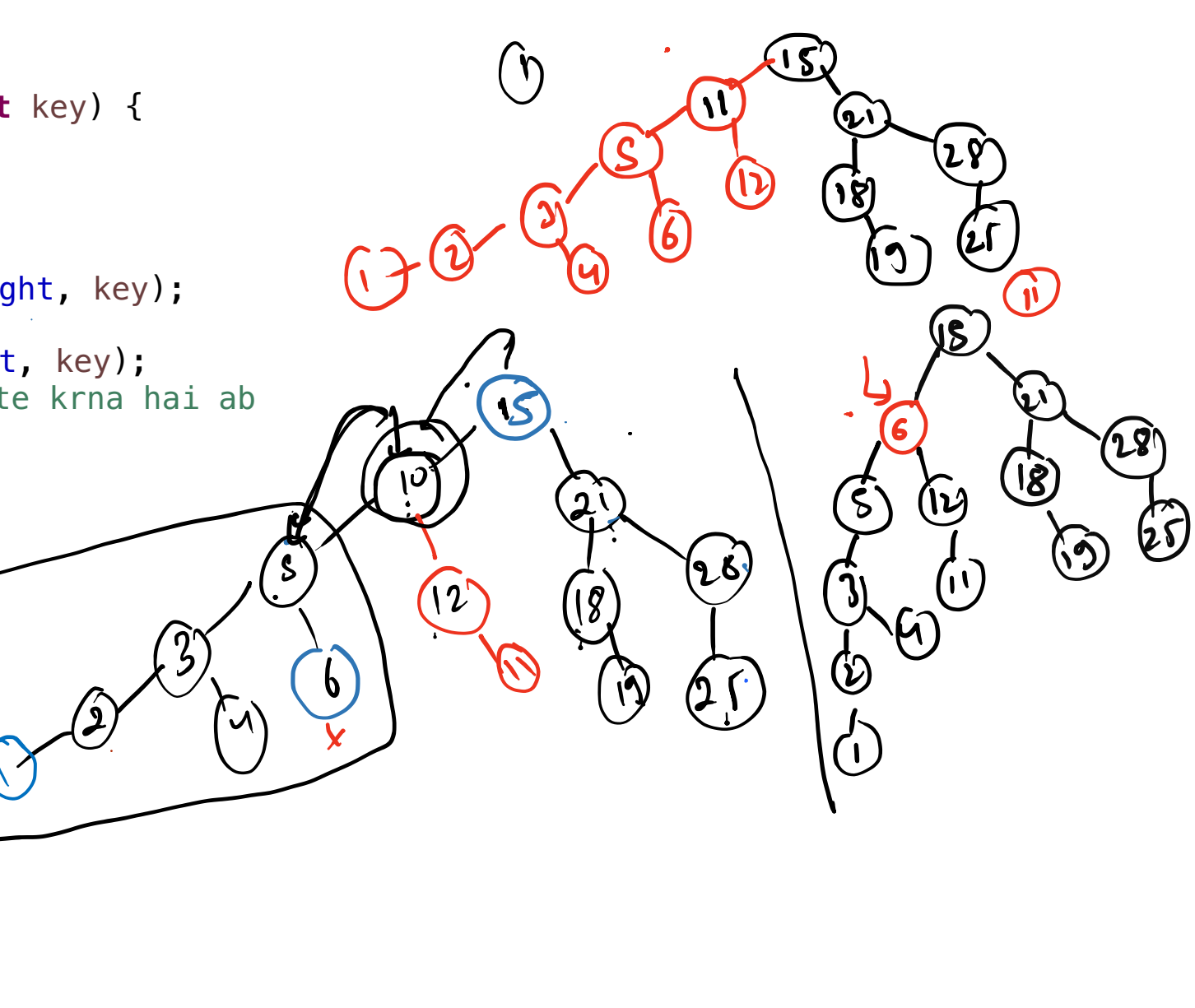

30
2
0-1
3-5
10
50
0,-1
0
1-1
3-3
4
5-5
20
40
60

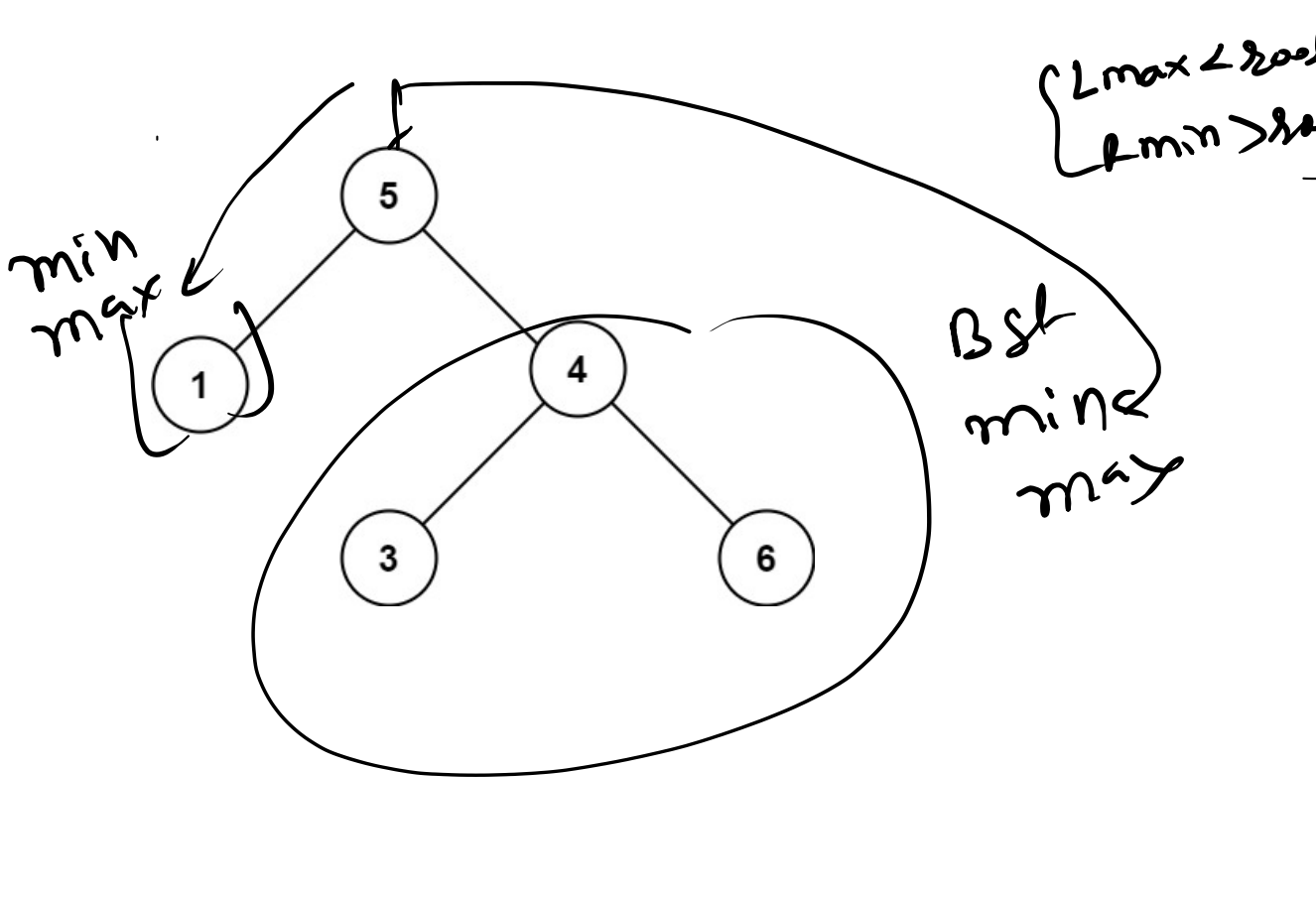
```
public TreeNode insertIntoBST(TreeNode root, int val) {
        if(root==null) {
            return new TreeNode(val);
        }

        if (root.val < val) {
            root.right = insertIntoBST(root.right, val);
        } else {
            root.left = insertIntoBST(root.left, val);
        }
        return root;
    }
```


8
15
28
10
21
28
5
12
18
28
3
6
13
19
25
2
4
1
8

```
public TreeNode deleteNode(TreeNode root, int key) {
        if (root == null) {
            return null;
        }
        if (root.val < key) {
            root.right = deleteNode(root.right, key);
        } else if (root.val > key) {
            root.left = deleteNode(root.left, key);
        } else {// data match kra hai delete krna hai ab
            if(root.left==null) {
                return root.right;
            }
            else if(root.right==null) {
                return root.left;
            }
            else {

            }
        }
        return root;
    }
```


15
6
21
5
12
18
28
3
11
19
25
2
4
1


1
15
11
5
21
3
6
12
18
28
1
2
4
19
25
11
15
10
21
8
12
18
28
3
6
11
19
25
2
4
1
15
6
21
5
12
18
28
3
11
19
25
2
4
1


5
min
max
1
4
3
6
Bst
min
max

{Lmax < root
Rmin > root}

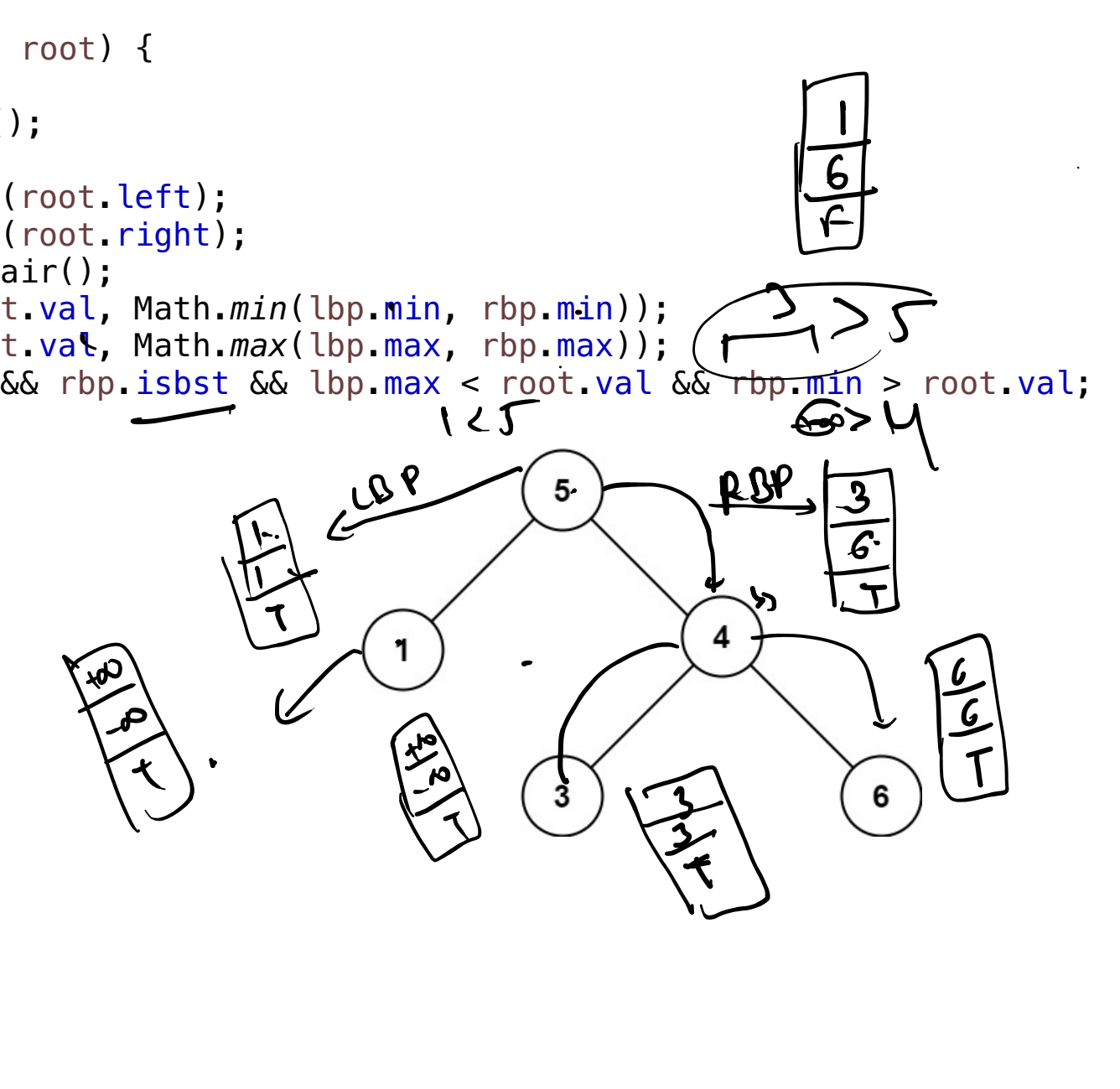
class Node{
long min = max
long max = min
boolean isBST

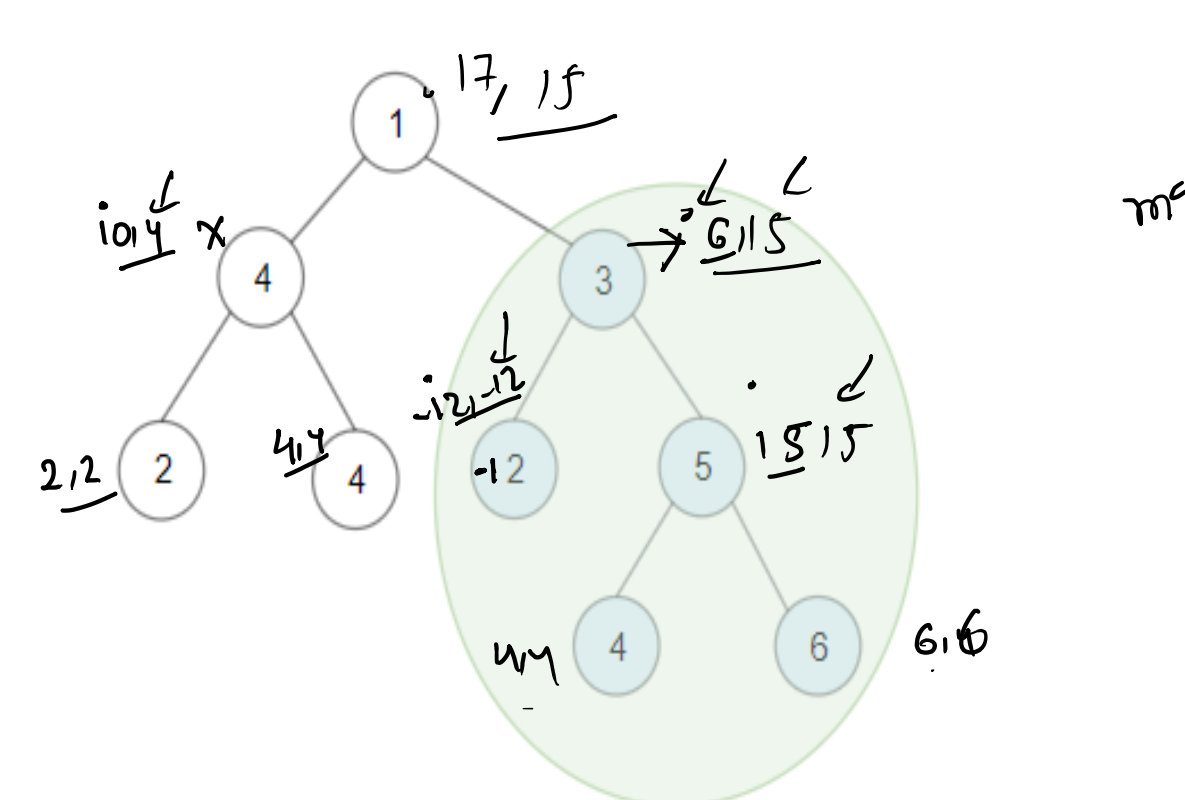
```
public BstPair ValidBST(TreeNode root) {
        if (root == null) {
            return new BstPair();
        }
        BstPair lbp = ValidBST(root.left);
        BstPair rbp = ValidBST(root.right);
        BstPair sbp = new BstPair();
        sbp.min = Math.min(root.val, Math.min(lbp.min, rbp.min));
        sbp.max = Math.max(root.val, Math.max(lbp.max, rbp.max));
        sbp.isbst = lbp.isbst && rbp.isbst && lbp.max < root.val && rbp.min > root.val;
        return sbp;
    }
```

1 < 5

3 > 5

3 > 4


1
6
F
LBP
5
RBP
3
6
T
1
1
T
1
4
+∞
-∞
T
+∞
-∞
T
3
6
3
3
T
6
6
T


1
17, 15
10, 4
4
3
6, 15
-12, -12
2, 2
2
4, 4
4
-12
5
15, 15
4, 4
4
6
6, 6

max (15, 4, 6)